

Découverte de la programmation Android

Le projet complet est disponible sur FramaGit.org/digicrafters/mini-rpg-android

Table des matières

1. Nathaniel Vaur Henel
2. Acte I - Introduction
3. Acte II - Mise en place
4. Acte III - Notions fondamentales
5. Acte IV - Construction du jeu
6. Acte V - Test & personnalisation libre
7. Acte VI - Bilan & ouverture

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Nathaniel Vaur Henel

L'auteur

Auteur

 Nathaniel Vaur Henel 🤘

- 👨‍🏫 Formateur
- 👨‍💻 Tech leader
- 🧉 Expert Java
- 💻 20 ans d'expérience en développement, principalement en Java
- 📜 Créateur de contenus pour dévs : dev-sensei.digicrafters.fr
- Discord : `@nathanielvaurhenel`



framagit.org/nathvh/me



dev-sensei.digicrafters.fr



linkedin.com/in/nathaniel-vaur-henel

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**



Acte I - Introduction

Bienvenue dans l’atelier Android

Android

Vous avez tous un smartphone. Mais savez-vous ce qui tourne dessus ? ✋

Android est le système d'exploitation mobile **le plus utilisé au monde** :

- 🌎 Plus de **3 milliards** d'appareils actifs dans le monde
- 📊 Environ **72 %** de parts de marché mondial sur les smartphones
- 🏪 Plus de **3 millions** d'applications disponibles sur le Google Play Store
- 🔧 Développé par Google, basé sur le noyau Linux, open-source

Chiffres 2024

> 🧔 Quand tu codes une appli Android, tu entres potentiellement dans des milliards de poches.

Kotlin : le langage officiel Android

Pourquoi apprendre Kotlin plutôt que Java pour Android ? ✋

Kotlin est le langage **officiellement recommandé par Google** depuis **2017** :

- ✅ **Moderne** : syntaxe claire et concise, moins de code pour faire la même chose
- ✅ **Sûr** : protection native contre les `NullPointerException`
- ✅ **100 % interopérable** avec Java
- ✅ **Adopté massivement** : plus de 60 % des apps Android professionnelles sont en Kotlin

```
val message = "Bienvenue, Adhakhan !"
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Construire une UI moderne Android

Comment construit-on les écrans d'une appli Android aujourd'hui ? ✋

Jetpack Compose est la **nouvelle façon officielle** de créer des interfaces Android :

- ✍️ **Tout en Kotlin** : l'interface est du code, comme le reste
- ♻️ **Réactif** : l'écran se met à jour automatiquement quand les données changent
- 🧩 **Composants réutilisables** : on assemble des blocs comme des LEGO

```kotlin
@Composable
fun MonEcran() {
    Text("Bonjour, monde !")
}
```

> 🧔 Compose, c'est comme du React ou du Flutter, mais pour Android, et en Kotlin.

Ce qu'on va construire aujourd'hui 🎮

Un mini-jeu **RPG tour par tour** complet, fonctionnel, sur Android :

- 🦸 **Créer son héros** : choisir un nom et une classe (Guerrier, Mage, Voleur, Archer)
- ⚔️ **Combattre des monstres** : Gobelin, Orc, Troll, Ogre… jusqu'au Dragon 🐉
- 📈 **Progresser** : gagner des PV et de l'attaque après chaque victoire
- 🏆 **Gagner ou perdre** : message de victoire finale ou écran de défaite avec relance

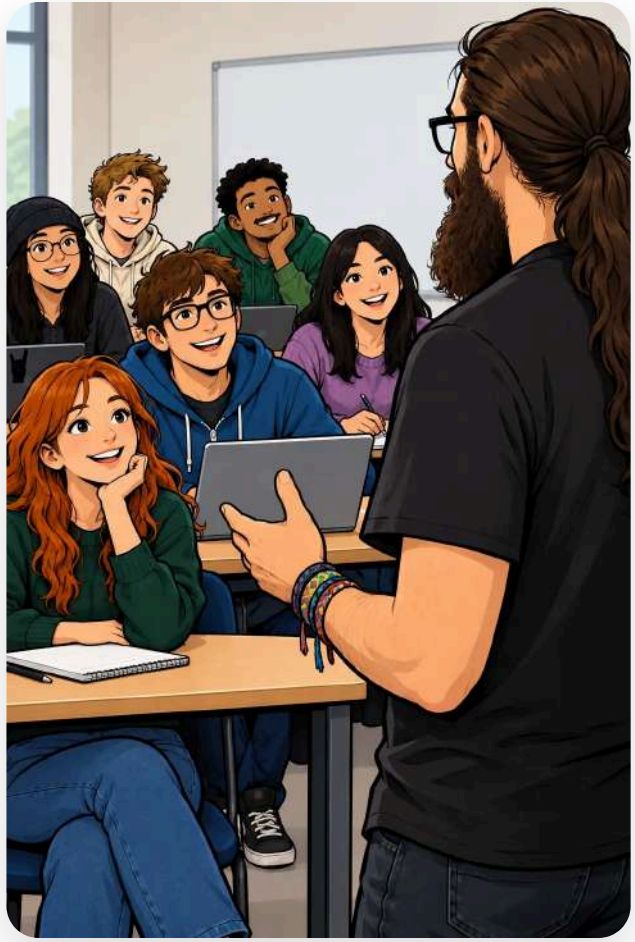
```
Héros : Adhakhan (Guerrier) - 250 PV - 15 ATK
VS
Monstre : Gobelin - 30 PV - 5 ATK
```

> 🧔 Dans 1 heure, vous repartez avec un vrai jeu qui tourne sur un vrai téléphone Android.

Questions
Acte I - Introduction

Vos questions ✋
Vos demandes d'ajout ✋

- ?





Acte II - Mise en place

Premier contact avec Android Studio 🛠️

Tour rapide d'Android Studio

À quoi sert chaque partie de l'interface qu'on a devant nous ? ✋

Android Studio est l'IDE officiel pour développer des applications Android :

- 📁 **Project** (panneau gauche) : l'arborescence de tous les fichiers du projet
- 🖍️ **Éditeur central** : là où on écrit le code Kotlin
- 📱 **Device Manager** (panneau droit) : pour gérer les émulateurs
- 🔨 **Toolbar** (en haut) : bouton ▶️ pour compiler et lancer l'appli
- 🦎 **Logcat** (en bas) : les logs de l'application en temps réel

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Créer un nouveau projet

🏋️ *C'est à votre tour de travailler* 🏋️

1. Ouvrir Android Studio
2. Cliquer sur "**New Project**"
3. Choisir le template "**Empty Activity**" 🧔 *Et surtout pas un autre !*
4. Renseigner les informations :
   - **Name** : `MiniRPG`
   - **Package name** : `je.decouvre.android.minirpg`
   - **Language** : `Kotlin`
   - **Minimum SDK** : `API 24` (Android 7.0)
5. Cocher "**Build configuration language : Kotlin DSL**"
6. Cliquer sur "**Finish**"

> 🧔 Le template "Empty Activity" avec Compose, c'est notre point de départ : une page blanche, prête à être remplie.

Structure du projet

Voici les fichiers importants dans l'arborescence :

```
app/
└── src/
    └── main/
        ├── java/com/decouverte/minirpg/
        │   └── MainActivity.kt   ← ✅ Notre fichier principal
        └── res/                  ← Images, icônes, ressources
```

Dans `MainActivity.kt`, repérez ces deux éléments clés :

```kotlin
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // 👈 Tout notre UI Compose sera écrit ici
        }
    }
}
```

 `setContent {}` : c'est la porte d'entrée de notre interface. Tout ce qui est dedans s'affiche à l'écran.

Lancer l’émulateur

🏋️ *C’est à votre tour de travailler* 🏋️

Lancez l’émulateur et vérifiez que tout fonctionne :

1. Cliquer sur "**Device Manager**" (icône téléphone & ordinateur en haut à droite)
2. Sélectionner l’émulateur disponible (ex : Pixel 6, API 34)
3. Cliquer sur ▶️ pour démarrer l’émulateur
4. Cliquer sur ▶️ **Run** dans la toolbar principale
5. Observer le résultat dans l’émulateur 👇

```
Hello Android!
```

⏳ *Le premier lancement peut être long…* ⏳

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

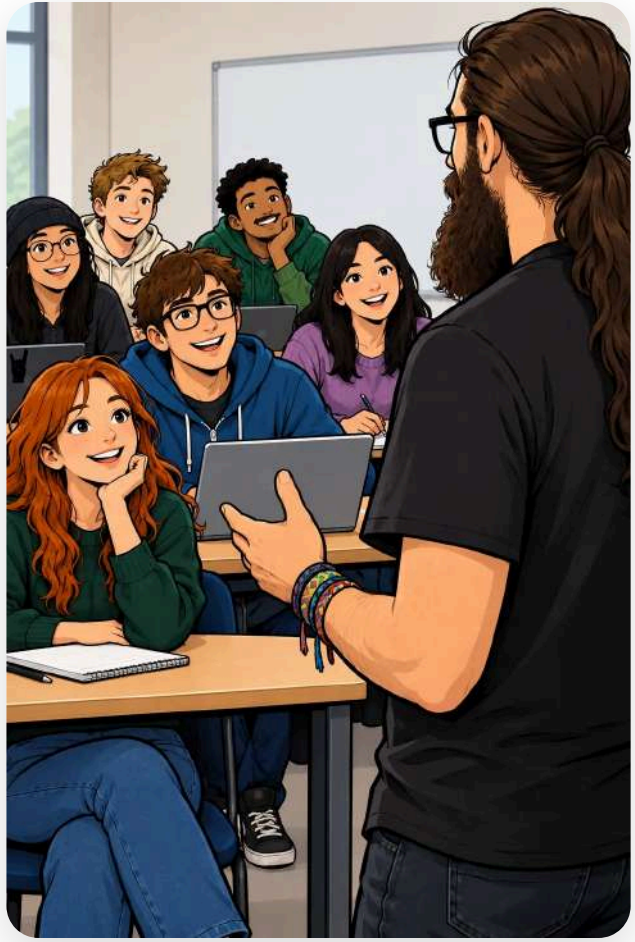
Questions

Acte II - Mise en place

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Acte III - Notions fondamentales

Kotlin & Jetpack Compose en 10 minutes ⚡



Kotlin, la langue du code

Les variables

Comment déclare-t-on une donnée en Kotlin ? ✋

En Kotlin, on distingue deux types de variables :

```
val nom = "Adhakhan"   // val = valeur fixe, ne peut pas changer
var pv = 250           // var = variable, peut être modifiée

pv = 80                // ✅ OK, pv est une var
nom = "Legolas"        // ❌ Erreur ! nom est un val
```

Mot-clé	Signification	Modifiable ?
`val`	value	❌ Non
`var`	variable	✅ Oui

> Par défaut, préférez toujours `val`. N'utilisez `var` que si la valeur doit vraiment changer.

Déclarer une fonction

Quelle est l'utilité d'une fonction ? ✋

Elle permet de découper le code en blocs réutilisables.
En Kotlin, une fonction se déclare avec le mot-clé `fun` :

```kotlin
fun direBonjour(nom: String): String {
    return "Bienvenue, $nom !"
}
// Appel de la fonction
val message = direBonjour("Adhakhan")
// message = "Bienvenue, Adhakhan !"
// > Le `$nom` dans la chaîne de caractères, c'est de la string interpolation. `${1 + 1}``pour les cas complexes
// > Kotlin remplace automatiquement la variable par sa valeur.
```

Anatomie d'une fonction :

```
fun  direBonjour  (nom: String)  : String  { ... }
 ↑       ↑              ↑             ↑
mot-clé  nom      paramètre     type de retour
```


Avec Kotlin, mettez du `fun` dans vos applis !

La logique conditionnelle

Comment prendre des décisions dans le code ? ✋

Le `if / else` en Kotlin fonctionne comme dans la plupart des langages, mais il peut aussi retourner une valeur :

```
var pv = 80

if (pv > 0) {
    println("Le héros est vivant !")
} else {
    println("Le héros est mort...")
}

// En Kotlin, if peut être une expression :
val etat = if (pv > 0) "Vivant" else "Mort"
```

On l'utilisera dans notre jeu pour :

- Vérifier si le héros ou le monstre a encore des PV ✅
- Gérer les fins de partie (victoire / défaite) 🏆 💀

> 🧔 En Kotlin, `if` n'est pas juste une instruction, c'est une expression qui peut renvoyer une valeur directement.

Objet de données simple

Comment représenter une entité du jeu (héros, monstre) ? ✋

Une `data class` est un moyen rapide de créer un objet structuré :

```
data class Monstre(val nom: String, var pv: Int, val attaque: Int)

// Créer un monstre
val gobelin = Monstre(nom = "Gobelin", pv = 30, attaque = 5)

// Accéder aux propriétés
println(gobelin.nom)      // "Gobelin"
println(gobelin.pv)       // 30
println(gobelin.attaque)  // 5
```

Récap Kotlin

Voici le code complet qu'on vient de voir :

```kotlin
val nom = "Adhakhan"
var pv = 250

fun direBonjour(nom: String): String {
    return "Bienvenue, $nom !"
}

data class Monstre(val nom: String, var pv: Int, val attaque: Int)
```

Ces 4 notions, `val/var`, `fun`, `if/else`, `data class`, couvrent **80 % du Kotlin** qu'on va utiliser dans notre jeu.

> 🧔 Vous n'avez pas besoin de tout connaître Kotlin pour construire quelque chose de fonctionnel.
> Commencez petit, comprenez ce que vous écrivez.



Jetpack Compose, la langue du dessin

Une fonction pour les dessiner tous

Comment Compose sait-il que cette fonction est un composant d'interface ? ✋

En Jetpack Compose, une fonction d'interface est annotée avec `@Composable` :

```
@Composable
fun MonComposant() {
    Text("Bonjour depuis Compose !")
}
```

Règles à retenir :

- Une fonction `@Composable` **ne retourne rien**
- Elle **décrit** ce qui doit s'afficher à l'écran
- Elle peut appeler d'autres fonctions `@Composable`
- Elle **ne peut être appelée** que depuis une autre fonction `@Composable`

> 🧔 Pensez à un `@Composable` comme à une brique LEGO : une seule ne fait pas grand-chose, mais assemblées entre elles, elles construisent tout.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Et dans l'IHM les organiser

Comment disposer plusieurs composants sur l'écran ? ✋

`Column` et `Row` sont les deux composants de mise en page fondamentaux :

```
@Composable
fun MiseEnPage() {
    Column {                        // ↕️ Empile verticalement
        Text("Ligne 1")
        Row {                       // ↔️ Dispose horizontalement
            Text("Gauche")
            Text("Droite")
        }
    }
}
```

```
╔═══════════════╗
║  Ligne 1      ║   ← Column
║  Gauche Droite ║  ← Row à l'intérieur
╚═══════════════╝
```

 Column = flex-direction: column. Row = flex-direction: row. Si vous avez fait du CSS, c'est exactement ça.

`Text`, `Button`, `TextField`, `DropdownMenu`

Les composants de base qu'on utilisera dans notre jeu :

```
// Afficher du texte
Text("PV du héros : 250")
// Un bouton cliquable
Button(onClick = { /* action */ }) {
    Text("Attaquer !")
}
// Saisie de texte
TextField(
    value = nom,
    onValueChange = { nom = it },
    label = { Text("Nom du héros") }
)
// Menu déroulant (pour choisir la classe)
DropdownMenu(
    expanded = menuOuvert,
    onDismissRequest = { menuOuvert = false }
) {
    DropdownMenuItem(text = { Text("Guerrier") }, onClick = { /* ... */ })
}
```

Gérer l'état de l'UI

Comment faire en sorte que l'interface se mette à jour automatiquement ? ✋

C'est **le concept clé** de Compose :

```
@Composable
fun MonComposant() {
    var compteur by remember { mutableStateOf(0) }

    Column {
        Text("Valeur : $compteur")
        Button(onClick = { compteur++ }) {
            Text("Cliquer")
        }
    }
}
```

- `mutableStateOf(0)` : crée un état observable initialisé à `0`
- `remember { }` : mémorise cet état entre les recompositions
- Quand `compteur` change → Compose **redessine automatiquement** les composants qui l'utilisent

> 🤓 En Compose, quand l'état change, l'UI se redessine toute seule. C'est la *recomposition*.

Récap Compose

```
@Composable
fun MonComposant() {
    var compteur by remember { mutableStateOf(0) }

    Column {
        Text("Valeur : $compteur")
        Button(onClick = { compteur++ }) {
            Text("Cliquer")
        }
    }
}
```

Ce petit exemple contient **tout le modèle mental** de Compose :

- Un `@Composable` décrit l'UI
- `Column` et `Row` organise les éléments
- `Text` et `Button` sont les composants de base
- `remember` + `mutableStateOf` gèrent l'état réactif

> Maîtrisez ce schéma, tout le reste n'est que variation sur ce même principe.

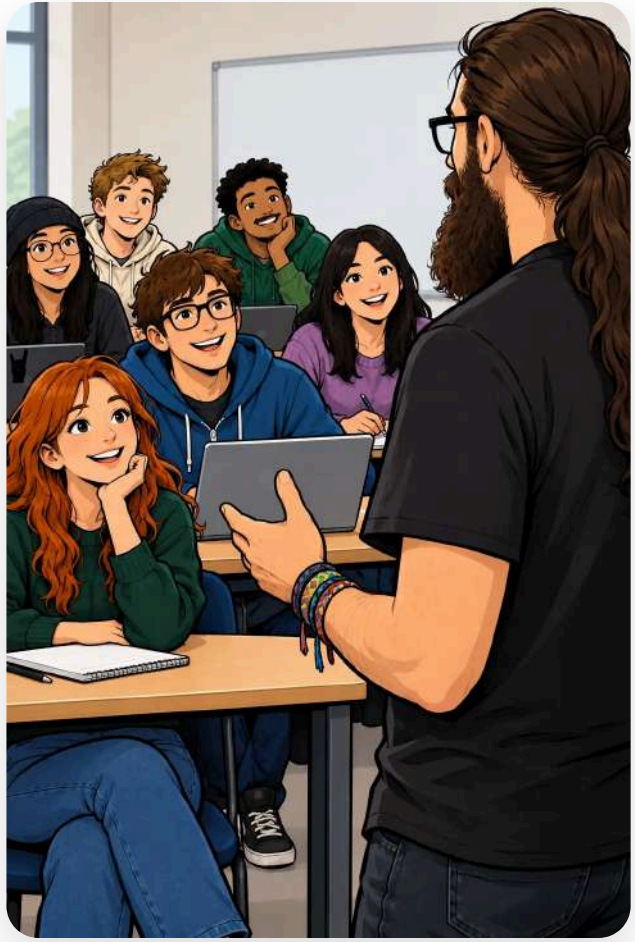
Questions

Acte III - Notions fondamentales Kotlin & Compose

Vos questions ✋

Vos demandes d'ajout ✋

- ?



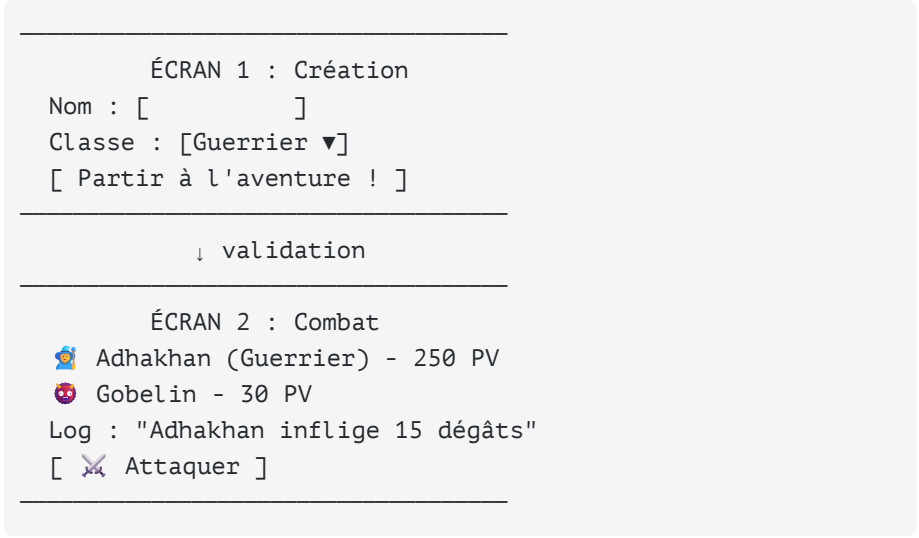


Acte IV - Construction du jeu

On code le mini RPG 🎮⚔️

Vue d’ensemble du jeu

Avant de coder, visualisons ce qu’on va construire :



4 étapes pour y arriver :

- **4.1** : Les données du jeu
- **4.2** : L’écran de création du personnage
- **4.3** : L’écran de combat
- **4.4** : La progression et les fins de partie

> 🧔 Avant de construire quoique ce soit, on fait un plan.
> C’est pareil en dév 🛠️

4.1 - Les données du jeu

La structure d'un monstre

Comment représenter un monstre avec ses caractéristiques ? ✋

On utilise une `data class`, exactement ce qu'on a vu en Phase 3 :

```
data class Monstre(val nom: String, var pv: Int, val attaque: Int)
```

Notez que `pv` est un `var` car les points de vie **changent** pendant le combat. `nom` et `attaque` sont des `val` car ils ne changent jamais.

> 🧔 `val` ou `var` dans une `data class`, la règle est la même : tout ce qui change est `var`, tout ce qui est fixe est `val`.

4.1 - La liste des monstres

🏋️ *C'est à votre tour de travailler* 🏋️

Ajoutez cette liste dans votre `MainActivity.kt`, en dehors de la classe :

```
data class Monstre(val nom: String, var pv: Int, val attaque: Int)

val monstres = listOf(
    Monstre("Gobelin",  30,   5),
    Monstre("Orc",      60,  12),
    Monstre("Troll",   100,  18),
    Monstre("Ogre",    150,  25),
    Monstre("Dragon",  220,  35)
)
```

La difficulté **progresse** : chaque monstre est plus costaud que le précédent.
Le Dragon en fin de liste est le **boss final** 🐉

> 🧔 `listOf()` crée une liste **immuable**, on ne peut pas y ajouter ou supprimer d'éléments. C'est parfait pour une liste de monstres qui ne change jamais.

4.1 - Les stats du héros

Les caractéristiques de départ du héros et sa progression :

```
// Stats initiales
var pvHeros    = 250   // Points de vie de départ
var atkHeros   = 15    // Puissance d'attaque de départ
// Progression après chaque monstre vaincu
// +50 PV  et  +5 ATK
```

Les 4 classes jouables, chacune avec son identité :

Classe	Identité
Guerrier	⚔️ Corps à corps
Mage	🔮 Magie
Voleur	🗡️ Discrétion
Archer	🏹 Distance

> 🧔 Dans cet atelier, les classes sont cosmétiques : elles changent le texte du bouton, pas les stats.

4.2 - L'écran de création du personnage

🏋️ *C'est à votre tour de travailler* 🏋️
On a besoin de **trois états** pour stocker les choix du joueur :

https://framagit.org/digicrafters/mini-rpg-android/-/blob/main/Snippets/src/main/kotlin/EcranCreation.kt

```
@Composable
fun EcranCreation(onJouer: (String, String) -> Unit) {
    var nomHeros by remember { mutableStateOf("") }
    var classeChoisie by remember { mutableStateOf("Guerrier") }
    var menuOuvert by remember { mutableStateOf(false) }

    // Suite sur la prochaine slide...
}
```

- `nomHeros` : ce que le joueur tape dans le TextField
- `classeChoisie` : la classe sélectionnée dans le DropdownMenu
- `menuOuvert` : pour afficher ou cacher le menu déroulant

Sur chaque mot en rouge, faire un `ALT` + `ENTER` afin d'*importer* le composant.


Chaque information qui peut changer et qui doit impacter l'UI devient un état, c'est le réflexe *Compose* à développer.

4.2 - Le TextField du nom

🏋️ *C'est à votre tour de travailler* 🏋️
Juste après la déclaration des 3 `var` dans la fonction `EcranCreation`

```
Column(modifier = Modifier.padding(16.dp)) {
    // Saisie du nom
    Text("Créez votre héros", style = MaterialTheme.typography.headlineMedium)
    Spacer(modifier = Modifier.height(16.dp))
    TextField(
        value = nomHeros,
        onValueChange = { nomHeros = it },
        label = { Text("Nom du héros") }
    )
    Spacer(modifier = Modifier.height(16.dp))
}
```

4.2 - Le DropdownMenu de la classe

🏋️ *C'est à votre tour de travailler* 🏋️
À la suite du `TextField` précédent, dans la `Column`

```
// Menu déroulant pour la classe
Box {
    Button(onClick = { menuOuvert = true }) {
        Text("Classe : $classeChoisie")
    }
    DropdownMenu(
        expanded = menuOuvert,
        onDismissRequest = { menuOuvert = false }
    ) {
        listOf("Guerrier", "Mage", "Voleur", "Archer").forEach { classe ->
            DropdownMenuItem(
                text = { Text(classe) },
                onClick = { classeChoisie = classe; menuOuvert = false }
            )
        }
    }
}
Spacer(modifier = Modifier.height(16.dp))
```

Le `Box` permet de positionner le `DropdownMenu` juste en dessous du bouton.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

4.2 - Le bouton "Partir à l'aventure !"

🏋️ *C'est à votre tour de travailler* 🏋️
Ajoutez le bouton de validation sous le menu déroulant :

```
    Button(
        onClick = {
            if (nomHeros.isNotBlank()) {
                onJouer(nomHeros, classeChoisie)
            }
        },
        enabled = nomHeros.isNotBlank()
    ) {
        Text("Partir à l'aventure !")
    }
```

- `enabled = nomHeros.isNotBlank()` : le bouton est **désactivé** tant que le nom est vide
- `onJouer(nomHeros, classeChoisie)` : on **remonte** les données au composant parent pour passer à l'écran de combat

> 🧔 Valider les données **avant** d'avancer, c'est du bon sens. Et c'est aussi ce qu'on fait dans les vrais projets.

4.2 - Gérer le passage entre les écrans

🏋️ *C'est à votre tour de travailler* 🏋️

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            var enCombat by remember { mutableStateOf(false) }
            var nomHeros by remember { mutableStateOf("") }
            var classeHeros by remember { mutableStateOf("") }
            var pvHeros by remember { mutableStateOf(250) }
            var atkHeros by remember { mutableStateOf(15) }
            var indexMonstre by remember { mutableStateOf(0) }
            var logs by remember { mutableStateOf(mutableListOf<String>()) }
            if (!enCombat) {
                /* EcranCreation */
            } else {
                /* EcranCombat à venir */
            }
        }
    }
}
```

 Un simple `if/else` pour une navigation simple. Pour des apps plus complexes, on utiliserait Jetpack Navigation.

4.2 - Ajouter l'écran de création

🏋️ *C'est à votre tour de travailler* 🏋️

Il faut lui passer tous les états dont il a besoin en paramètre.

```
    EcranCreation(
        onJouer = { nom, classe ->
            nomHeros = nom
            classeHeros = classe
            enCombat = true
        })
```

4.3 - L’écran de combat

L’écran de combat affiche trois zones distinctes :

```
┌──────────────────────────┐
│ 🦸 Héros                 │ ← Bloc Héros
│ Adhakhan (Guerrier)      │
│ PV : 250  ATK : 15       │
├──────────────────────────┤
│ 👹 Monstre               │ ← Bloc Monstre
│ Gobelin : PV : 30        │
├──────────────────────────┤
│ Log de combat            │ ← LazyColumn
│ > Adhakhan attaque...    │
│ > Gobelin riposte...     │
├──────────────────────────┤
│ [ ⚔️ Attaquer ]          │ ← Bouton d'action
└──────────────────────────┘
```

> 🧔 Découper l’UI en blocs visuels distincts avant de coder, c’est du UI design de base, et ça évite de se perdre.

4.3 - Le bloc Héros

🏋️ *C'est à votre tour de travailler* 🏋️
Créez un composable dédié pour le héros :

```
@Composable
fun BlocHeros(nom: String, classe: String, pv: Int, attaque: Int) {
    Card(modifier = Modifier.fillMaxWidth().padding(8.dp)) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("🦸 Votre héros", style = MaterialTheme.typography.titleMedium)
            Text("$nom - $classe")
            Text("PV : $pv  |  ATK : $attaque")
        }
    }
}
```

On utilise une `Card` pour encadrer visuellement chaque bloc, car c'est un composant Material Design prêt à l'emploi.

> 🧔 Extraire chaque bloc dans son propre `@Composable`, c'est le réflexe "composant réutilisable". Votre futur vous dira merci.

4.3 - Le bloc Monstre

🏋️ *C'est à votre tour de travailler* 🏋️
Créez le composable pour le monstre actuel :

```
@Composable
fun BlocMonstre(monstre: Monstre) {
    Card(modifier = Modifier.fillMaxWidth().padding(8.dp)) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("👹 Ennemi", style = MaterialTheme.typography.titleMedium)
            Text(monstre.nom)
            Text("PV : ${monstre.pv}  |  ATK : ${monstre.attaque}")
        }
    }
}
```

On passe directement l'objet `Monstre` en paramètre, car on accède à ses propriétés avec la notation pointée.

4.3 - Le log de combat avec LazyColumn

🏋️ *C'est à votre tour de travailler* 🏋️

On utilise un `LazyColumn` qui permettra d'afficher une colonne d'objets :

```
@Composable
fun LogCombat(logs: List<String>) {
    LazyColumn(
        modifier = Modifier
            .fillMaxWidth()
            .padding(8.dp)
    ) {
        items(logs.size) { index ->
            Text("> ${logs[index]}", style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

- `LazyColumn` ne charge que les éléments **visibles**, donc c'est performant même avec des centaines de lignes
- On passe une `List<String>` qu'on alimentera à chaque tour de combat

> 🧔 "Lazy" = paresseux. `LazyColumn` ne dessine que ce qui est à l'écran, c'est la bonne pratique pour toute liste en Compose.

4.3 - La mécanique de combat

La formule de combat introduit une part d'aléatoire :

```
Dégâts du héros   = attaque_hero   + random(-10% attaque_hero,   +10% attaque_hero)
Dégâts du monstre = attaque_monstre + random(-20% attaque_monstre, +20% attaque_monstre)
```

En Kotlin :

```kotlin
//🏋️ _C'est à votre tour de travailler_ 🏋️
//Ajouter cette méthode à votre code

import kotlin.random.Random // tout en haut du fichier

fun calculerDegats(attaque: Int, variancePct: Int): Int {
    val variance = (attaque * variancePct / 100.0).toInt()
    return attaque + Random.nextInt(-variance, variance + 1)
}
// Utilisation, pour exemple
val degatsHeros   = calculerDegats(atkHeros,       10)  // +/-10%
val degatsMonstre = calculerDegats(monstre.attaque, 20)  // +/-20%
```



Le random, c'est ce qui rend le jeu imprévisible, et donc amusant. Sans ça, chaque combat serait identique.

4.3 - Le bloc Attaque

🏋️ *C'est à votre tour de travailler* 🏋️

```
@Composable
fun BlocAttaque(
    atkHeros: Int,
    monstreActuel: Monstre,
    onAttaquer: (degatsHeros: Int, degatsMonstre: Int) -> Unit
) {
    Button(
        onClick = {
            val degatsHeros = calculerDegats(atkHeros, 10)
            val degatsMonstre = calculerDegats(monstreActuel.attaque, 20)
            onAttaquer(degatsHeros, degatsMonstre)
        },
        modifier = Modifier.fillMaxWidth().padding(8.dp)
    ) {
        Text("⚔️ Attaquer")
    }
}
```

On remonte les dégâts calculés au composant parent via `onAttaquer`, c'est le pattern de communication standard en Compose.

4.3 - Assembler l'écran de combat

🏋️ *C'est à votre tour de travailler* 🏋️

Maintenant qu'on a tous les blocs, assemblons-les dans l'écran principal :

```
@Composable
fun EcranCombat(
    nomHeros: String,
    classeHeros: String,
    pvHeros: Int,
    atkHeros: Int,
    monstreActuel: Monstre,
    logs: List<String>,
    onAttaquer: (Int, Int) -> Unit
) {
    Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {
        BlocHeros(nomHeros, classeHeros, pvHeros, atkHeros)
        BlocMonstre(monstreActuel)
        Box(modifier = Modifier.weight(1f)) {LogCombat(logs)} // LazyColumn prend toute la place restante
        BlocAttaque(atkHeros, monstreActuel, onAttaquer)
    }
}
```

Chaque bloc est un `@Composable` indépendant, l'écran de combat ne fait que les **composer** ensemble.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

4.3 - Intégrer l'écran de combat

🏋️ *C'est à votre tour de travailler* 🏋️
Dans votre `MainActivity`, appelez `EcranCombat` dans le bloc `else` :

```
if (!enCombat) {
    EcranCreation(/* ... */)
} else {
    // Créer un état pour les PV actuels du monstre
    var pvMonstreActuel by remember(indexMonstre) {
        mutableStateOf(monstres[indexMonstre].pv)
    }
    EcranCombat(
        nomHeros = nomHeros,
        classeHeros = classeHeros,
        pvHeros = pvHeros,
        atkHeros = atkHeros,
        monstreActuel = monstres[indexMonstre].copy(pv = pvMonstreActuel),
        logs = logs,
        onAttaquer = { degatsHeros, degatsMonstre -> /* La logique de combat sera ajoutée dans les prochaines slides */}
    )
}
```

- `remember(indexMonstre)` : réinitialise les PV du monstre. Sans ça, le nouveau monstre hériterait des PV de l'ancien.

4.3 - Gérer la logique de combat

🏋️ *C'est à votre tour de travailler* 🏋️

Maintenant, complétez le callback `onAttaquer` dans votre `MainActivity` :

```
EcranCombat(
    // tous les paramètres
    onAttaquer = { degatsHeros, degatsMonstre ->
        // 1. Application des dégâts
        pvMonstreActuel -= degatsHeros
        pvHeros -= degatsMonstre

        // 2. Mise à jour du log
        logs = logs.toMutableList().apply {
            add("$nomHeros inflige $degatsHeros dégâts")
            add("${monstres[indexMonstre].nom} riposte pour $degatsMonstre dégâts")
        }
    }
)
```

🧔 Le pattern `callback`, un composant enfant ne modifie rien directement, il prévient juste le parent. C'est un flux de données unidirectionnel.

4.4 - Monstre vaincu : progression

🏋️ *C'est à votre tour de travailler* 🏋️

Après chaque monstre vaincu, on applique les bonus et on passe au suivant.

Dans le bloc `EcranCombat`, modifiez le callback `onAttaquer` après la mise à jour du log :

```
EcranCombat(
    // ... paramètres
    onAttaquer = { degatsHeros, degatsMonstre ->
        // [...]
        // 3. ✅ AJOUTEZ ICI : Vérification victoire sur le monstre
        if (pvMonstreActuel <= 0) {
            logs = logs.toMutableList().apply {
                add("✅ ${monstres[indexMonstre].nom} est vaincu !")
                add("📈 +50 PV, +5 ATK pour $nomHeros !")
            }
            pvHeros += 50
            atkHeros += 5
            indexMonstre++
        }
    }
)
```


L'index de liste, c'est notre "pointeur de niveau" : il avance d'un cran à chaque boss vaincu.

4.4 - Héros vaincu : défaite...

🏋️ *C'est à votre tour de travailler* 🏋️

Dans le callback `onAttaquer`, après la vérification du monstre vaincu :

```
EcranCombat(
    // ... paramètres
    onAttaquer = { degatsHeros, degatsMonstre ->
        // [...]
        // 4. ✅ AJOUTEZ ICI : Vérification défaite du héros
        if (pvHeros <= 0) {
            logs = logs.toMutableList().apply {
                add("💀 $nomHeros est mort au combat...")
            }
        }
    }
)
```

4.4 - Héros vaincu : … et recommencer

🏋️ *C'est à votre tour de travailler* 🏋️ Dans le composable `EcranCombat`, modifier le `Column` pour ajouter un bouton conditionnel après `BlocAttaque` :

```
Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {
    BlocHeros(nomHeros, classeHeros, pvHeros, atkHeros)
    BlocMonstre(monstreActuel)
    LogCombat(logs)
    // Bouton Attaquer si le héros est vivant
    if (pvHeros > 0) {
        BlocAttaque(nomHeros, atkHeros, monstreActuel, onAttaquer)
    } else {
        // Bouton Recommencer si le héros est mort
        Button(
            onClick = { /* sera géré par un callback onRecommencer */ },
            modifier = Modifier.fillMaxWidth().padding(8.dp)
        ) {
            Text("🔄 Recommencer")
        }
    }
}
```

 Game over, c'est juste un reset d'état. En Compose, c'est aussi simple que ça : on remet les vars à zéro, et l'UI se redessine toute seule.

4.4 - Ajouter le bouton Recommencer

🏋️ *C'est à votre tour de travailler* 🏋️

Pour que le bouton "Recommencer" fonctionne, **modifiez la signature d'** `EcranCombat` pour ajouter un callback :

```
@Composable
fun EcranCombat(
    // paramètres existant
    onRecommencer: () -> Unit  // ✅ AJOUTEZ ce paramètre
) {
    // ... Column existante avec le if/else pour le bouton
    if (pvHeros > 0) {
        BlocAttaque(nomHeros, atkHeros, monstreActuel, onAttaquer)
    } else {
        Button(
            onClick = { onRecommencer() },  // ✅ Appelez le callback
            modifier = Modifier.fillMaxWidth().padding(8.dp)
        ) {
            Text("🔄 Recommencer")
        }
    }
}
```

4.4 - Et son Callback

🏋️ *C'est à votre tour de travailler* 🏋️
Dans la `MainActivity`, passez le callback :

```
EcranCombat(
    // ... autres paramètres
    onRecommencer = {
        enCombat = false
        pvHeros = 250
        atkHeros = 15
        indexMonstre = 0
        logs = mutableListOf()
    }
)
```

4.4 - Dragon vaincu : victoire finale 🏆

🏋️ *C'est à votre tour de travailler* 🏋️

Quand `indexMonstre` dépasse la taille de la liste, tous les monstres sont vaincus.
Dans le `setContent`, modifier la structure `if/else` pour gérer trois états :

```
setContent { /* paramètres inchangés */ }
    // ✅ ÉTAT 1 : Écran de création
    if (!enCombat) {
        /* EcranCreation inchangé */
    }
    // ✅ ÉTAT 2 : Victoire finale
    else if (indexMonstre >= monstres.size) {
        Column(
            modifier = Modifier.fillMaxSize().padding(16.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text("🏆 Félicitations, $nomHeros !",
                style = MaterialTheme.typography.headlineLarge)
            Text("Tu as vaincu le Dragon et sauvé le royaume !")
            Text("🐉 L'aventure se termine ici... pour l'instant.")
            Spacer(modifier = Modifier.height(24.dp))
            Button(onClick = {
                enCombat = false
                pvHeros = 250
                atkHeros = 15
                indexMonstre = 0
                logs = mutableListOf()
            }) {
                Text("🔄 Recommencer")
            }
        }
    }
    // ✅ ÉTAT 3 : Combat en cours
    else {
        /* EcranCombat inchangé */
    }
}
```

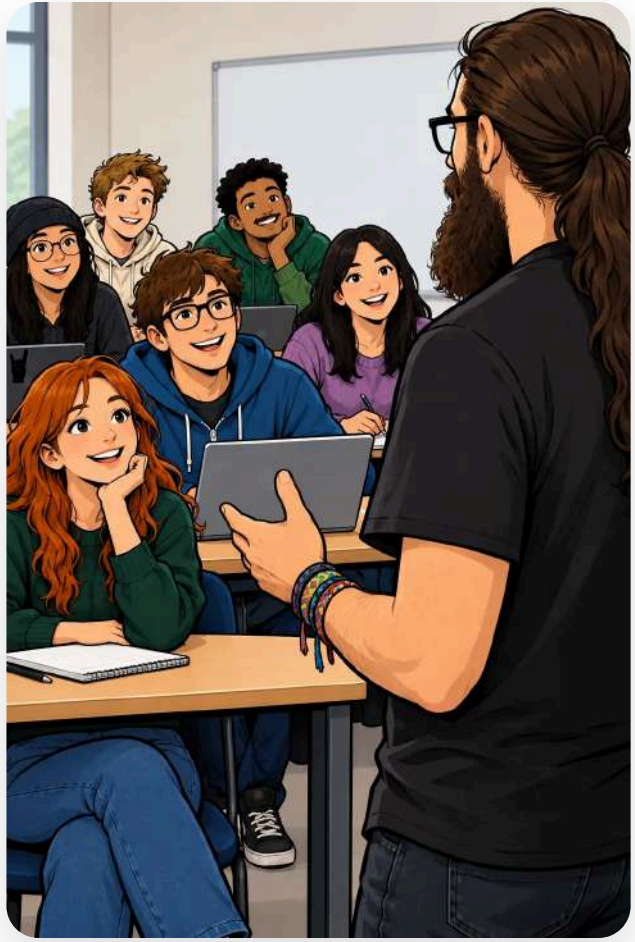
Questions

Acte IV - Construction du jeu

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Acte V - Test & personnalisation libre

À vous de jouer ! 🕹️

Tester le jeu sur l’émulateur

🏋️ *C’est à votre tour de travailler* 🏋️
Lancez votre application et vérifiez que tout fonctionne :

1. Cliquer sur ▶️ **Run** dans Android Studio
2. Attendre que l’application se lance sur l’émulateur
3. Tester le parcours complet :
   - ✅ Saisir un nom de héros 🦸
   - ✅ Choisir une classe dans le menu déroulant
   - ✅ Cliquer sur "Partir à l’aventure !"
   - ✅ Attaquer plusieurs fois jusqu’à vaincre le Gobelin
   - ✅ Vérifier que le bonus de progression s’applique (+15 PV, +5 ATK)
   - ✅ Tenter de vaincre le dragon 🐉
   - ✅ Tester de recommencer 🔄

> 🧔 Tester son code soi-même avant de le montrer à quelqu’un, c’est une règle importante pour un développeur.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Défi pour les rapides 🏆

Et si le bouton "Attaquer" changeait selon la classe choisie ? ✋

Objectif : adapter le texte du bouton Attaquer en fonction de la classe du héros.

Classe	Texte du bouton
Guerrier	`⚔️ Frapper`
Mage	`🔮 Lancer un sort`
Voleur	`🗡️ Poignarder`
Archer	`🏹 Tirer une flèche`

Indice : utilisez un `when` (l'équivalent Kotlin du `switch`) :

```
val texteBouton = when (classeChoisie) {
    "Guerrier" -> "⚔️ Frapper"
    "Mage"     -> "🔮 Lancer un sort"
    "Voleur"   -> "🗡️ Poignarder"
    "Archer"   -> "🏹 Tirer une flèche"
    else       -> "⚔️ Attaquer"
}

Button(onClick = { /* ... */ }) {
    Text(texteBouton)
}
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

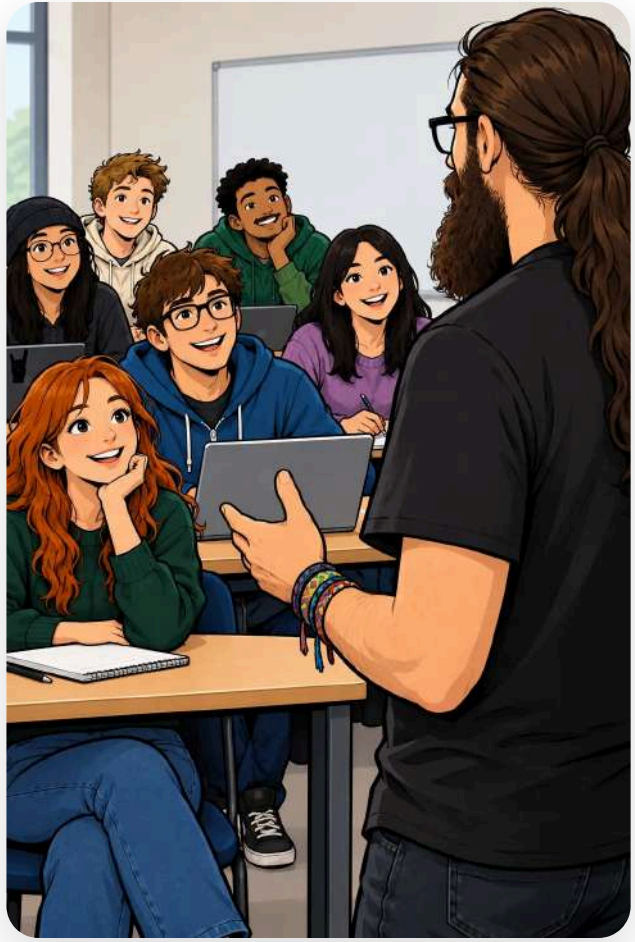
Questions

Acte V - Test & personnalisation libre

Vos questions ✋

Vos demandes d'ajout ✋

- ?





Acte VI - Bilan & ouverture

Ce qu’on a construit, ce qu’on pourrait faire 🚀

Ce qu'on a vu aujourd'hui

Notion	Contexte dans le jeu
`val` / `var`	Stats du héros et des monstres
`data class`	Structure `Monstre`
`List` / `listOf`	La liste des 5 monstres
`fun`	Fonctions de combat
`if / else`	Fin de combat, progression
`Random`	Dégâts aléatoires
`@Composable`	Toute l'interface
`remember` + `mutableStateOf`	État du jeu
`Column` / `Row`	Mise en page
`TextField` / `Button` / `DropdownMenu`	Widgets de l'UI
`LazyColumn`	Log de combat

 En environ 1 heure, vous avez construit les bases d'un jeu.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Ce qu'on pourrait ajouter

Si vous aviez une semaine de plus, qu'est-ce qu'on pourrait faire ?

Voici les extensions naturelles de ce mini-jeu :

- 🔊 **Sons et musique** : `MediaPlayer` ou `SoundPool` pour les coups et la victoire
- 🎞️ **Animations** : `AnimatedVisibility`, transitions entre les écrans
- 💾 **Sauvegarde** : `DataStore` pour conserver la progression entre les sessions
- 🗺️ **Plusieurs écrans** : navigation avec Jetpack Navigation Compose
- 🌐 **API** : récupérer les monstres depuis un serveur, classement en ligne
- 🎨 **Thème personnalisé** : Material Design 3, couleurs, typographies
- 💉 **Tests** : tester les fonctions de combat avec JUnit

> 🧔 Ce jeu est une graine. Avec les bonnes connaissances, vous pouvez en faire quelque chose de professionnel.

La suite en cours

Ce qu'on a effleuré aujourd'hui, une formation de développeur logiciel vous permet de le maîtriser :

- 📱 **Développement Android complet** : architecture, navigation, persistance, API REST
- 🌐 **Web & Mobile** : React, Vue, Angular, APIs, bases de données
- ☁️ **Cloud & DevOps** : Docker, CI/CD, déploiement
- 🤝 **Travail en équipe** : Git, méthodologies agiles, code review
- 🏢 **Projets réels** : alternance, projets clients, portfolio

En peu de temps, vous passez de `Hello Android!` à développeur junior capable de rejoindre une équipe professionnelle.

> 🧔 Le plus dur, c'est le premier pas. Vous venez de le franchir.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Et maintenant ? 🚀

L'atelier est fini… mais l'aventure ne fait que commencer !

Le code complet est disponible sur FramaGit : https://framagit.org/digicrafters/mini-rpg-android

Pourquoi aller le récupérer ?

- 💾 **Gardez une trace** de ce que vous avez construit aujourd'hui
- 🎨 **Personnalisez** : changez les couleurs, les textes, les stats
- 🔧 **Expérimentez chez vous** : ajoutez des sons, des animations, de nouveaux monstres
- 🏆 **Relevez les défis** : implémentez les extensions proposées
- 📚 **Apprenez en pratiquant** : c'est en codant qu'on devient développeur

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

Continuez l'aventure 🎮

Quelques idées pour aller plus loin **dès ce soir** :

Niveau débutant 🌱 *On expérimente* (1-2h)

- Changer les noms des monstres
- Ajouter un 6ème monstre "Boss Secret"
- Modifier les stats de départ du héros
- Changer les émojis et les textes

Niveau intermédiaire 🌿 *On améliore* (3-5h)

- Faire que chaque classe ait des stats différentes
- Ajouter des potions de soin
- Créer un système de critique (double dégâts)
- Ajouter des images pour les monstres

Niveau avancé 🌳 (plusieurs jours)

- Sauvegarder la progression avec DataStore
- Ajouter des sons et de la musique
- Créer un système de loot
- Créer un système d'équipement (armes, armures)
- Faire un écran des meilleurs scores
- Rencontrer des monstres aléatoires classés par niveaux
- etc.

> 🧔 Mon meilleur ami a commencé le dév au lycée en codant un mod pour le jeu vidéo *Half-Life* .

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

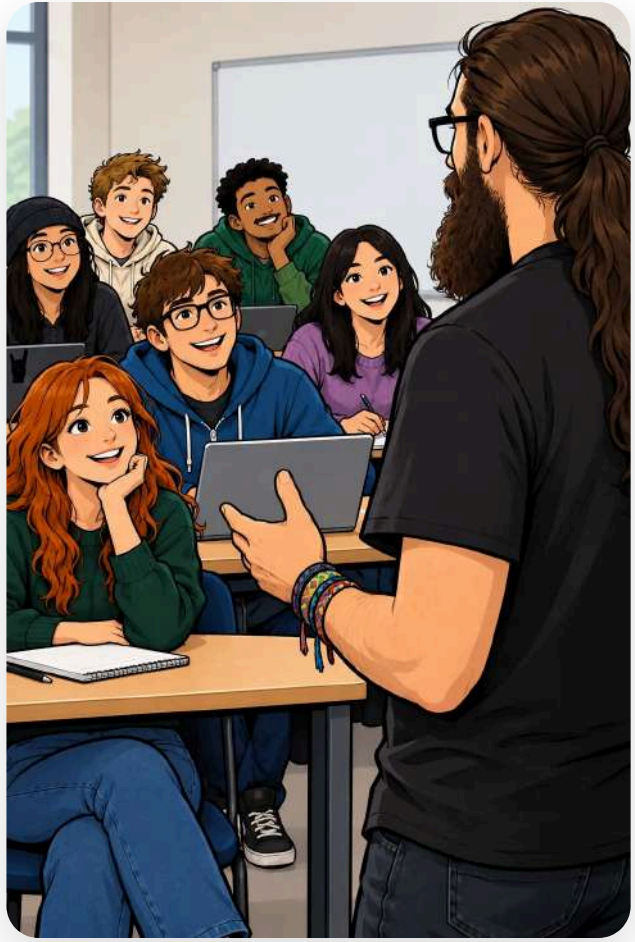
Questions

Acte VI - Bilan & ouverture

Vos questions ✋

Vos demandes d'ajout ✋

- ?



🤝 Merci de ton attention ! 🤝

Reste curieux 🧐 et amuse-toi ! 🎮 🎉

Ce cours t'a été présenté par

 Moi-même, Nathaniel Vaur Henel 🤘

 mon site de partage autour du dév : dev-sensei.digicrafters.fr

 mon Framagit perso, GitLab éthique, libre & français : framagit.org/nathvh/me

 mon GitHub perso, mais inactif : github.com/Nathaniel-Vaur-Henel

 mon LinkedIn, pour suivre mon parcours pro : linkedin.com/in/nathaniel-vaur-henel

 mon Discord pro, pour échanger ou me poser des questions : `@nathanielvaurhenel`